

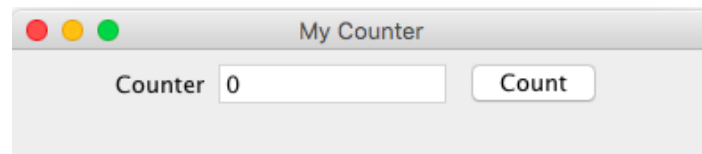
Tutorial - Week 09 5COSC019W – Object Oriented Programming – Java

Event Listener, File Handling

18-19 November

GUI application

- 1) Write a Swing GUI application called Counter, as shown in the figure below. Every time the Count button is clicked, the counter value should increase by 1.



The user interface has 3 JComponent:

- JLabel
- JButton
- JTextField

The Swing Components are to be added onto the ContentPane of the top-level container JFrame. You can retrieve the ContentPane via the method `getContentPane()` from a JFrame.

Note: Swing Components are kept in package `javax.swing`. They begin with a prefix "J", e.g., JButton, JLabel, JFrame.

```
import java.awt.*;           // Using AWT's layouts
import javax.swing.*;        // Using Swing components and containers
```

Write a class Counter that extends JFrame:

```
public class Counter extends JFrame{

    private JLabel lblCount;        // Declare component JLabel
    private JTextField tfCount;     // Declare component JTextField
    private JButton btnCount;       // Declare component JButton
    private int count = 0;          // counter's value

    // Constructor to setup UI components and event handlers
    public Counter () {
        // sets layout to FlowLayout, which arranges
        // Components from left-to-right, then top-to-bottom.
        Container cp = getContentPane();
        cp.setLayout(new FlowLayout());

        lblCount = new JLabel("Counter"); // Construct component Label
        add(lblCount);                    // "super" Frame adds Label

        tfCount = new JTextField(count + "", 10); // Construct component TextField
        tfCount.setEditable(false);               // read-only
        add(tfCount);                             // "super" Frame adds TextField

        btnCount = new JButton("Count"); // Construct component Button
        add(btnCount);                   // "super" Frame adds Button
    }
}
```

Instance variables

Implement Constructor

Get content Pane and set the layout

Add component to the content pane

Event Handling

The **first step** is to write an inner class that implements `EventListener`.

- An inner class called `MyListener` is defined to be used as the listener for the `ActionEvent` fired by the Button `btnCount`. Since `MyListener` is an `ActionEvent` listener, it has to implement the `ActionListener` interface and provide an implementation for the `actionPerformed()` method declared in the interface.
- Even if instance variables `tfCount`, and `count` are private, the inner class `MyListener` has access to them. This is the reason why an inner class is used instead of an ordinary outer class.

In the same file, implement the listener:

```
private class MyListener implements ActionListener {
    @Override
    public void actionPerformed(ActionEvent evt) {
        ++count;
        tfCount.setText(count + "");
    }
}
```

Every time an `ActionEvent` is generated the count variable is incremented by 1 and the value is displayed in the GUI

The **second step** is to register the listener to the `JComponent`: in your code, you need to add the following lines in the `Counter` constructor

```
MyListener handler = new MyListener();
btnCount.addActionListener(handler);
```

- `btnCount` is the **source** object that fires `ActionEvent` when clicked
- The source add "handler" instance as an `ActionEvent` listener, which provides an `ActionEvent` handler called `actionPerformed()`.
- Clicking `btnCount` invokes `actionPerformed()`.

2) Modify the previous code in order to include two additional buttons for counting down and resetting the count value.

Hints:

- Add the other two `JButtons` to the content pane and register the Event Listener
- You can use `evt.getSource()` to retrieve which component has fired the event

```
public void actionPerformed(ActionEvent evt) {

    // checking which is the source that fired the event
    if (evt.getSource() == btnCount) {
        ++count;
        tfCount.setText(count + "");
    } else if (evt.getSource() == btnCountDown) {
        --count;
        tfCount.setText(count + "");
    } else if (evt.getSource() == btnReset) {
        count = 0;
        tfCount.setText(count + "");
    }
}
```

JUnit Exercise

6) For this exercise, you can use the **Date** class that was provided in tutorial week 03. You can download the NetBeans project from BB or from the following link:

https://learning.westminster.ac.uk/ultra/courses/97941_1/outline/edit/document/4971557_1?courseId=97941_1&view=content

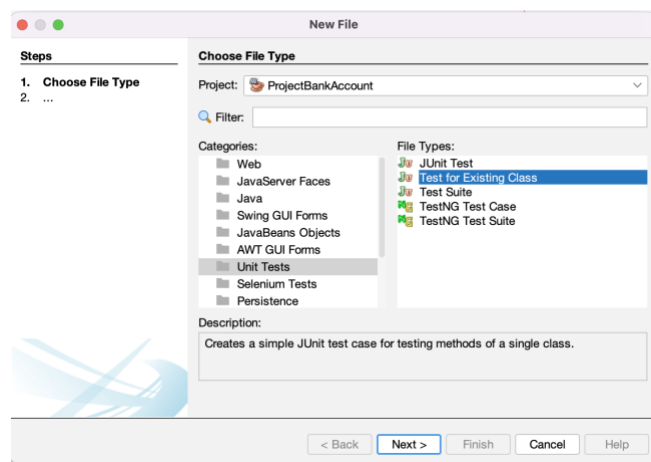
Your task is to write JUnit tests to test the following methods in the class Date:

- Constructor
- Set and get year methods
- Set and get month methods
- Set and get day methods

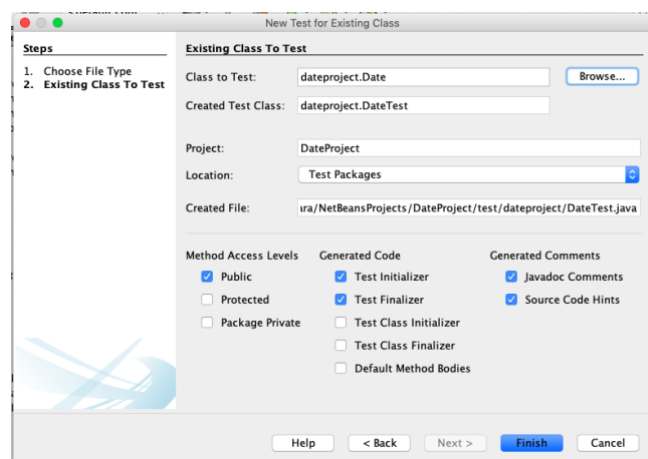
Some guidance on creating unit tests:

Writing Java and Test Classes Using NetBeans IDE:

1) Select the Java project "DateProject". Right click on the selected Java project. Select **New -> Other**. Select **Unit Tests** and then **Test for Existing Class**



1) Choose the class to test (in our case Date as we want to test the methods from this class) and tick or untick the check boxes as shown in the figure below. Press Finish.



The test class will be generated. You can keep only the first part, which corresponds to the constructor, the setUp and tearDown. You can delete all the other methods as we will build our tests.

```
public class DateTest {
```

```

    public DateTest() {
    }

    @BeforeEach
    public void setUp() {
    }

    @AfterEach
    public void tearDown() {
    }
}

```

@BeforeEach is an annotation that tells JUnit to run the setUp method before each test.

@AfterEach is an annotation to define a tearDown method, which runs after each test is completed. If it's empty, means no specific cleanup is needed.

Since we want to test methods on a Date object, let's create an instance of the class Date that will be used in each test method:

```

public class DateTest {

    private Date date;

    public DateTest() {
    }

    @BeforeEach
    public void setUp() {
    }

    @AfterEach
    public void tearDown() {
    }
}

```

We will instantiate the object Date inside the setUp() method. This ensure that a new Date instance is created before every test, with some initial values:

```

public class DateTest {

    private Date date;

    public DateTest() {
    }

    @BeforeEach
    public void setUp() {

        date = new Date(18, 11, 2024);
    }

    @AfterEach
    public void tearDown() {
    }
}

```

2) Testing the constructor

```

// test the constructor
@Test // indicates that is a test
public void testConstructor() {
    assertEquals(18, date.getDay());
    assertEquals(11, date.getMonth());
    assertEquals(2024, date.getYear());
}

```

This method checks if the Date object's day, month and year attributes are correctly set by the constructor.

assertEquals compares the expected value (e.g., 18) with the actual value returned by the getter method `doctor.getDay()`. If they are equal, the test passes; if not, it fails.

3) Testing the Getter and Setter for Day

```
@Test
public void testSetAndGetDay() {
    date.setDay(20);
    assertEquals(20, date.getDay());
}
```

This test checks the `setDay` and `getDay` methods.

`date.setDay(20)` sets the day to be 20, and `assertEquals` confirms that this value is correctly stored and returned by `date.getDay()`;

4) Implement by yourself the tests for the Getter and Setter methods for month and year

5) In the **Date** class implement the method `addDays` with the following signature:

```
public void addDays(int days)    // advances by days
```

This method advances the date by the number passed in the parameter *days*.

Come up with unit tests to check that the `addDays` method works properly.

```
/**
 * Test of addDays method, of class Date.
 */
@Test
public void testAddDays() {

    // add here your tests
    Date d = new Date(12, 2, 2019);

    d.addDays(4);
    assertEquals(2019, d.getYear()); // expected
    assertEquals(2, d.getMonth());  // value should
    assertEquals(16, d.getDay());    // be at LEFT

}
}
```

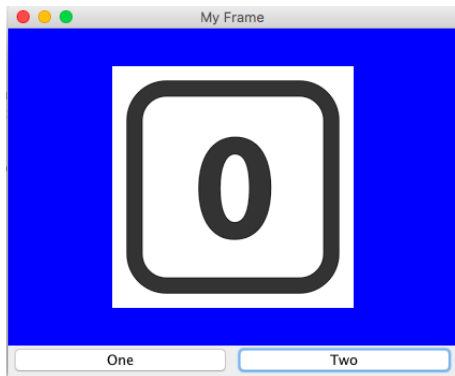
6) Write a test that checks boundary values. For example, check if the method `addDays` works as expected, adding days that wrap to the next month (Example: 28/12/2024 ->(add 4 days)->1/01/2025)

Further material and Extra Exercises that you can do in your own time at home

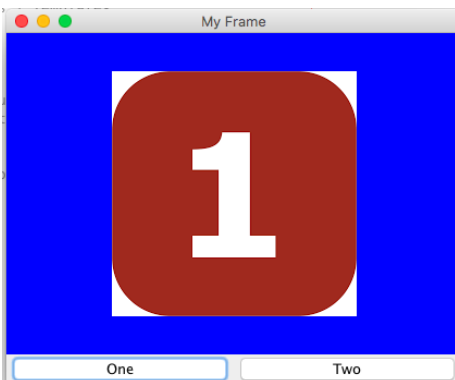
Show Images

Write a Swing GUI application that shows three different images (download the images from BB). The JFrame has the following components:

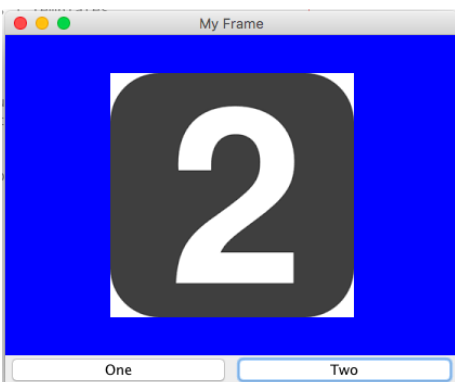
- JLabel (the image is displayed on this label)
- Two JButtons



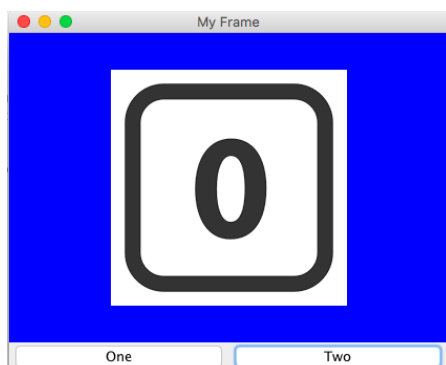
a) When you **run the application**, you will display a default image that represents a zero.



b) When you **click on "One" button** the image will change and it will display a one



c) When you **click on "Two" button** the image will change and it will display a two



d) When you **click on the image** it will return to display the default zero image.

Solution Show images:

You can download the images at the following link: [Images](#)

Implement a class called `ShowImageFrame` that extends `JFrame`. Write the instance variables and the constructor for this class. You need to:

- Set the layout in order to have the `JLabel` in the center and the two buttons on the bottom
- Set a default image to the `JLabel`
- Add listeners to your `JComponents`

```
public class ShowImageFrame extends JFrame{
```

```
    JButton btnOne;  
    JButton btnTwo;  
    JLabel lblImage;
```

} Instance variables

```
    ImageIcon imageIconZero;  
    ImageIcon imageIconOne;
```

} We represent our images with an `ImageIcon` object.
(`ImageIcon` class paints Icons from Images)

```
    ImageIcon imageIconTwo;
```

Constructor

```
public ShowImageFrame() {
```

```
    Container cp = getContentPane();  
    // set layout in the main panel  
    cp.setLayout(new BorderLayout());  
    cp.setBackground(Color.blue);
```

```
    // create a panel with two buttons in a grid  
    JPanel p1 = new JPanel();  
    p1.setLayout(new GridLayout(1,2));  
    //create and add buttons  
    btnOne = new JButton("One");  
    p1.add(btnOne);
```

```
    btnTwo = new JButton("Two");  
    p1.add(btnTwo);
```

```
    // create a label with a default image  
    imageIconZero = new ImageIcon(getClass().getResource("zero.png"));  
    lblImage = new JLabel(imageIconZero, JLabel.CENTER);
```

```
    //add the jlabel on the top  
    cp.add(lblImage, BorderLayout.CENTER);
```

```
    //add the button on the bottom  
    cp.add(p1, BorderLayout.SOUTH);
```

```
    // instantiate the other two images  
    imageIconOne = new ImageIcon(getClass().getResource("one.png"));  
    imageIconTwo = new ImageIcon(getClass().getResource("two.png"));
```

```
    // add event handler  
    MyListener myListener = new MyListener();  
    btnOne.addActionListener(myListener);  
    btnTwo.addActionListener(myListener);
```

```
    // add mouse listener to the jLabel  
    lblImage.addMouseListener(myListener);
```

```
}
```

- We retrieve the image reading the png file from the resource folder and we save it as `ImageIcon` object.
- We pass the `ImageIcon` object as argument to `JLabel` constructor

- We add an action listener to button One and button Two
- We add a mouse listener to the `JLabel`

- 1) Implement a listener to handle the `ActionEvent` and the `MouseEvent`. Read the following code and explain why `MyListener` extends `MouseAdapter` and implements `ActionListener`.

```
private class MyListener extends MouseAdapter implements ActionListener{

    public void actionPerformed(ActionEvent evt) {
        String btnLabel = evt.getActionCommand();

        // event.getActionCommand() returns the button's label
        if (btnLabel.equals("One")) {
            lblImage.setIcon(imageIconOne);

        } else if (btnLabel.equals("Two")) {
            lblImage.setIcon(imageIconTwo);
        }
    }

    public void mouseClicked(MouseEvent evt){
        lblImage.setIcon(imageIconZero);
    }
}
```

- 2) Write the main class to display your `ShowImageFrame`:

```
public static void main(String[] args) {
    // Invoke the constructor by allocating an anonymous instance
    ShowImageFrame myFrame = new ShowImageFrame();

    myFrame.setSize(600, 400);          // "super" Frame sets initial size
    myFrame.setTitle("My Frame");        // "super" Frame sets title
    myFrame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    myFrame.setVisible(true);           // show "super" Frame
}
```